

Booklet Creator Integration Plan

Summary: Task 2.

Generated: 2026-01-13 04:45 UTC | Ideas Extracted: 27

What Happened

Task 2.3: Integration with WAFT Tools [] Consider integration with `DocumentBuilder` : Add `.booklet()` method to `DocumentBuilder` Or standalone function: `waft.booklet.rearrange()` [] Integration with `Binder` system (optional): Rearrange binder pages for booklet printing [] Document integration patterns.

Skills Learned/Improved: Planning: Analyzed feature requirements Analysis: Understood algorithm and dependencies Architecture: Designed integration approach.

Knowledge Gained: Booklet printing page rearrangement algorithm PyPDF2 to pypdf migration patterns WAFT PDF tool ecosystem structure Integration patterns for new tools.

Feature Goal: Create a functional prototype that integrates booklet-creator's page rearrangement algorithm into WAFT's PDF tool ecosystem.

Source Repository: `chongchonghe/booklet-creator` Language: Python Dependencies: `PyPDF2=3.0.1` (old version) `numpy` Core Algorithm: Rearranges PDF pages for booklet printing Output: Creates `.booklet.pdf` file with rearranged pages.

The booklet-creator uses a specific page rearrangement algorithm: For an 8-page document, rearranges to: 4, 5, 6, 3, 2, 7, 8, 1 Designed for Tabloid (11x17) printing with: 2 pages per sheet Short-edge binding Normal 'Z' layout Handles page ranges (start/end pages) Can print page order without rearranging.

PDF Libraries: ☒ `pypdf>=3.0.0` (modern version, already in dependencies) ☒ `numpy` (likely available, need to verify) ☐ `PyPDF2` (old, deprecated - needs migration to `pypdf`).

Existing PDF Tools: `src/waft/document_builder.py` - Uses `pypdf.PdfReader` `src/waft/re-dactor.py` - Uses `pypdf.PdfReader`, `PdfWriter` `tools/pdf_binder_organizer/` - Uses `pypdf` for PDF manipulation `src/waft/binder.py` - Binder system for combining PDFs.

Task 1.1: Create Tool Directory ☐ Create `tools/booklet_creator/` directory ☐ Add `__init__.py` ☐ Create `README.md` with usage instructions.

Task 1.2: Migrate Code ☐ Copy `booklet.py` from repository ☐ Migrate `PyPDF2` → `pypdf` : Replace from `PyPDF2` import `PdfWriter` as `Writer`, `PdfReader` as `Reader` Replace with from `pypdf` import `PdfWriter`, `PdfReader` Update variable names if needed ☐ Update imports and API calls ☐ Test basic functionality.

Task 1.3: Dependency Check ☐ Verify `numpy` is available (add to dependencies if needed) ☐ Verify `pypdf>=3.0.0` is available (already in `pyproject.toml`) ☐ Update `requirements.txt` or `pyproject.toml` if needed.

Task 2.1: Create CLI Interface ☐ Create `cli.py` with Typer/Click interface ☐ Support command: `waft booklet <pdf_file> [options]` ☐ Options: `--start <page>` - Starting page number `--end <page>` - Ending page number `--print-pages` - Print page order without rearranging `--output <path>` - Custom output path ☐ Add help text and usage examples.

Task 3.1: Testing ☐ Create test PDFs (4, 8, 12, 16 pages) ☐ Test page rearrangement algorithm ☐ Verify output PDFs are valid ☐ Test edge cases: Minimum pages (4) Odd page counts Large documents Page ranges.

Additional Details

Task 3.2: Documentation [] Update `README.md` with: Usage examples Printer setup instructions Algorithm explanation Integration examples [] Add docstrings to all functions [] Create example script in `examples/`.

```
output = rearrange_for_booklet( pdf_path=Path("document.pdf"), start=1, end=None ) print(f"Booklet PDF created: {output}").
```

Task 4.2: Output Options [] Support custom output paths [] Preserve original filename with `.booklet` suffix (default) [] Option to overwrite or create new file.

Task 4.3: CLI Command Registration [] Add to main WAFT CLI if applicable [] Or document as standalone tool [] Update main README with tool reference.

[] Print test booklet on Tabloid paper [] Verify page order is correct when folded [] Verify binding works correctly.

Mitigation: Start with standalone tool (Option 1) Document integration patterns for future Keep API simple and focused.

Integration with DocumentBuilder Integration with Binder system Support for different paper sizes Support for different binding types GUI interface (optional).

Review plan with user Execute plan using `/evolve` command (spawns Being, executes workflow) Or execute manually following this plan Track execution through Being system Complete evolution cycle.

Planning Participation: Feature specification extracted Repository analyzed Dependencies identified Integration points found Comprehensive plan created.

Decision: Start with Option 1 (standalone tool) for prototype, can migrate to Option 2 later.

Task 2.2: Python API [] Create clean Python API in `booklet.py` :.

Task 3.3: Example Usage [] Create `examples/create_booklet.py` :.

✅ `pypdf>=3.0.0` (already in `pyproject.toml`) ⚠️ `numpy` (need to verify/add if missing).

```
doc = DocumentBuilder.field_guide(title="Guide", content="...") doc.save("guide.pdf").
```

Content Breakdown

Type	Count
Decisions	1
Insights	0
Actions	19
Concepts	6
Questions	1